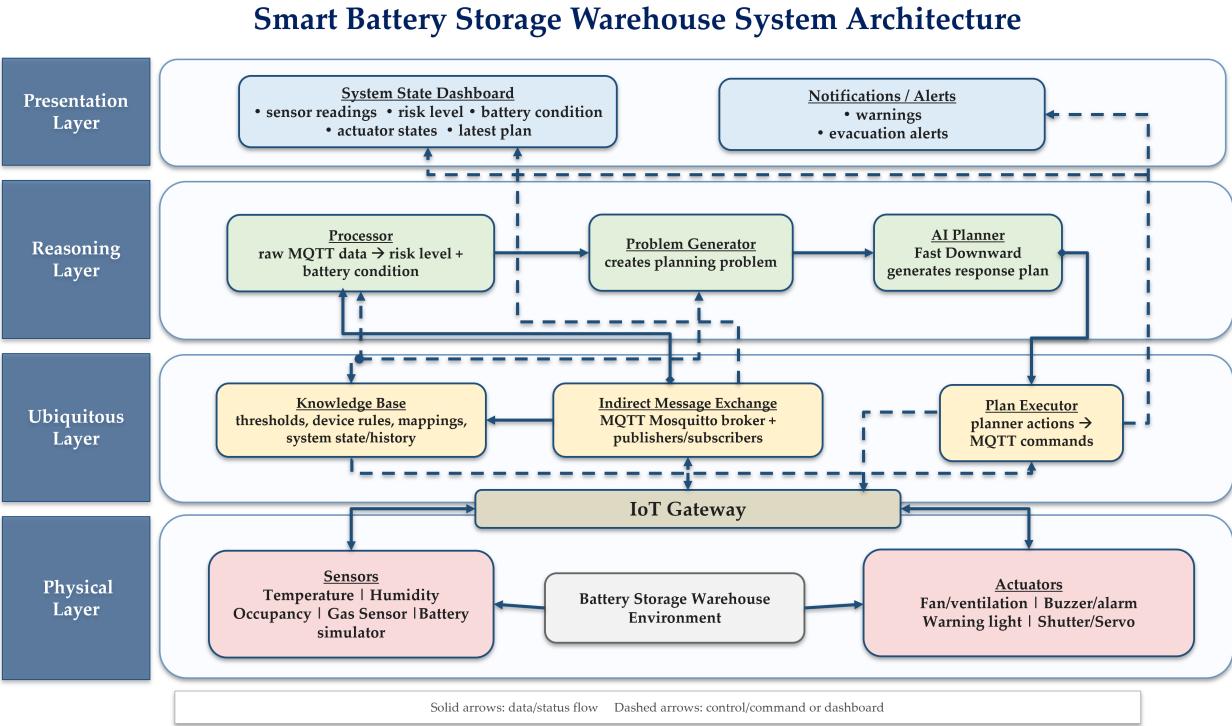


Project Information			
1. Student Name	Alaa Ali		Student ID 3822832
2. Student Name	Evan Trenchant		Student ID 3879319
3. Student Name	Salik Farrukh		Student ID 3807019
4. Student Name			Student ID
Group ID	29		
Project Title	Smart Battery Storage Warehouse		
Domain	☒ Smart building ☐ Autonomous vehicle		
Type	<i>Specify the type of the non-residential building (e.g., warehouse, offices) or the type of the autonomous ground vehicle (e.g., delivery van, taxi).</i> Non-residential smart building: battery storage warehouse.		
Project Description	Monitoring	<i>Specify what monitoring aspects your system includes.</i>	
		Temperature, humidity, gas, occupancy, risk level, and battery status.	
	Autotmation	<i>Specify what automation aspects your system includes.</i>	
		Risk detection, actuator control, dashboard updates, and phone alerts.	
	Objective	Ensure safety of the warehouse, and monitoring the battery conditions.	
<i>Give a link to code repository (e.g., GitHub, Bitbucket, etc.). See slide 56 in slides set "01 Overview and logistics".</i>			
Code Repository Link	https://github.com/AlaaAttia-H/Smart-Battery-Warehouse		

System Design			
System components	Components		Functionality
	☒	IoT	Read sensors, control physical actuators in RPi.
	☒	Context	Converts sensor data into risk & battery levels.
	☒	Problem generation	Creates PDDL problem from the current states.
	☒	Planning	Runs Fast Downward to generate response plan
	☒	Execution	Maps planner actions to MQTT actions.
	☒	Broker	Connects I/O, controllers, dashboard.
	☒	Knowledge base	Stores thresholds, zones, device data, actions.
	☒	Other. Specify what other logical components your system has.	
	Dashboard notifications		Node-RED visualises system state using MQTT. Ntfy sends manager alerts.

System Architecture Diagram

Include a clear diagram of your system architecture Ensure the diagram is a logical design and is not dependent on deployment and/or implementation.



System Distribution			
Number of Machines and Components Distribution	☒ 2+ machines		
	Specify the type of each machine (e.g., PC, cloud, or Raspberry Pi).		
	<table border="1"> <thead> <tr> <th>Type</th> </tr> </thead> <tbody> <tr> <td>Raspberry Pi, laptop/PC, manager PC and mobile device.</td> </tr> </tbody> </table>	Type	Raspberry Pi, laptop/PC, manager PC and mobile device.
	Type		
Raspberry Pi, laptop/PC, manager PC and mobile device.			
Specify what logical components are deployed on each machine.			
	<table border="1"> <thead> <tr> <th>Components</th> </tr> </thead> <tbody> <tr> <td>Raspberry Pi: IoT, MQTT broker and Dashboard. Laptop: AI Planner. Manager PC: Dashboard. Mobile Device: Push Notifs.</td> </tr> </tbody> </table>	Components	Raspberry Pi: IoT, MQTT broker and Dashboard. Laptop: AI Planner. Manager PC: Dashboard. Mobile Device: Push Notifs.
Components			
Raspberry Pi: IoT, MQTT broker and Dashboard. Laptop: AI Planner. Manager PC: Dashboard. Mobile Device: Push Notifs.			

IoT						
Sensors	Sensor (e.g., temperature)	Physical	Software-based	Human-based	Simulated	Virtual
	Humidity	☒	☐	☐	☐	☐
	Temperature	☐	☐	☐	☐	☐
	Gas sensor / MQ2	☐	☐	☐	☐	☐
	Occupancy/Push But	☐	☐	☐	☐	☐
	Other. Specify the remaining sensors and their types (if more than 4 sensors)					
	Battery status	Software-based simulated sensor. Battery percentage is entered and then published by the Raspberry Pi.				
Actuators	Actuator	Physical	Software-based	Human-based	Simulated	Virtual
	Fan	☒	☐	☐	☐	☐
	Buzzer	☐	☐	☐	☐	☐
	RGB LED	☐	☐	☐	☐	☐
	Servo/Door	☐	☐	☐	☐	☐
	Other. Specify the remaining actuators and their types (if more than 4 actuators)					
	Push Notifications Dashboard	Software-based actuator: Sends notifications to manager. Virtual actuator/visualisation action.				

System Integration	
Mechanism	☒ Publish-subscribe ☐ Message queue ☐ One-to-one
Messaging Middleware	☐ JMS ☐ ZeroMQ ☐ Apache Kafka ☐ RabbitMQ ☐ Redis ☐ IBM WebSphereMQ ☐ Apache Qpid ☒ Other: Mosquitto MQTT broker. ☒ MQTT ☐ DSS ☐ XMPP ☐ AMQP ☐ Other:
Mechanism	<i>Explain briefly which components of your system use indirect communication.</i> The Raspberry Pi hardware node, laptop AI planner, dashboard, and notification components communicate indirectly through the Mosquitto MQTT broker. Sensors publish data to MQTT topics. The laptop controller

Visualisation	
What is displayed	☒ Latest plan generated ☒ Current state. <i>Briefly specify what exactly:</i> Displays current sensor/system states and previous and current actions. ☒ User control. <i>Briefly explain:</i> There's a manual override option to the servo, to be used by the manager. ☐ Other. <i>Briefly explain:</i> The dashboard also shows sensor graphs, notificationsn sent.

AI Planning	
AI Planning Technique	☐ Classical planning ☐ HTN planning ☐ Other: Classical planning.
AI Planner	<i>Specify the name and link to the AI planner used in your project:</i> Fast Downward <i>Explain briefly why the chosen AI planner is appropriate for your project:</i> Because the system uses deterministic symbolic actions in PDDL. <i>Explain briefly the main components of your domain model:</i> The domain contains zones, a dashboard object, predicates for required actions and actuator states, and actions such as start-fan, stop-fan, activate-alarm, set-red-light, open-shutter, close-shutter, notify-manager, request-evacuation, and battery warning actions.
Domain Model	<i>Explain briefly what constitutes an initial state in your problem instances:</i> Initial State based on context (risk level, occupancy, battery condition). <i>Explain briefly what constitutes a goal in your problem instances:</i> Goal The desired safe response, ex: buzzer is on when HIGH risk.
Problem Instance	Problem instances are generated ☐ Once, before system starts. ☒ Whenever a state change happens. ☐ Other. <i>Explain briefly:</i> A new PDDL problem is generated whenever relevant context changes, such as risk level, gas status, occupancy, or battery condition.